

HW #1

CSEE W3827 - Fundamentals of Computer Systems Spring 2026

Prof. Rubenstein
Due Jan 29, 2026, 11:59pm

Topics: binary number representations (2-complement, 1-complement, floating point)

Note that this homework has 7 problems and is 2 pages long.

It's a bit long and tedious. Not all HW will be like this.

1. Assume an architecture where all numbers are to be represented using 8 bits. What are the (base 10) values of the 8-bit binary numbers when interpreted using (i) unsigned, (ii) signed magnitude, (iii) 1's complement, (iv) 2's complement form:
 - (a) 00110011
 - (b) 10000000
 - (c) 11111111
 - (d) 10011011
 - (e) 10001010
2. Convert the following (base 10) numbers to their binary representation using 8-bit (i) signed magnitude, (ii) 1's complement, (iii) 2's complement forms:
 - (a) -1
 - (b) -15
 - (c) -67
 - (d) -127
3. For each pair of x and y below, convert x and y to their 8-bit 2's-complement forms, **subtract** y from x (so you need to negate y and then add). Indicate whether an overflow occurred (and show clearly how you know), and convert the solution back to base 10 (even when an overflow occurred and the solution is wrong, i.e., your answer should fit in the 8-bit representation of that form.).
 - (a) $x = 10, y = -13$
 - (b) $x = 117, y = 35$
 - (c) $x = 117, y = -35$
 - (d) $x = -117, y = 35$
 - (e) $x = -117, y = 11$
4. Given the bit pattern 1010 1101 0001 0000 0000 0000 0010 , what does it represent, assuming that it is
 - (a) a two's complement integer?
 - (b) an unsigned integer?
 - (c) a single precision (32-bit) floating-point number?
5. Represent the following numbers, given here in binary form, as floating point numbers using IEEE 754 floating-point standard representation:
 - (a) 11010.1110
 - (b) -11011.10
 - (c) 0.000101
 - (d) -1.010101
 - (e) 10000000010

6. Below is a “weird” encoding for a 3-bit wordsize that encodes only 0 and prime values. Suppose we ask a 3-bit computer to add all possible pairs of values and return the result (in the same encoding, of course). Indiciate which pairs of numbers when summed together produce an overflow.

Bit sequence	value
000	0
001	1
010	2
011	3
100	5
101	7
110	11
111	13

7. **OPTIONAL** Challenge Question (with exam-like “twist”): Given a k -bit wordsize, explain which statements are true, and which ones are false, and why:
- It is possible to construct a representation that ranges between integer values n_1 and n_2 where $n_2 = n_1 + 2^k - 1$. For instance, unsigned has $n_1 = 0, n_2 = 2^k - 1$ and 2’s-complement has $n_1 = -2^{k-1}$ and $n_2 = 2^{k-1} - 1$. But can it be done, for example, with $n_1 = 12$, or $n_1 = -3$ What restrictions are there on n_1 as a function of k ?
 - For what values of n_1 will the BAA algorithm produce a correct result (modulo 2^k)? Why? (I’d probably give more hints for this question on an exam)
 - Hard question: Come up with a general algorithm to detect overflow. Note that both unsigned and 2’s-complement have easy methods to detect overflow (i.e., for unsigned, check the highest order carry, for 2’s-complement, compare the two highest order carries. So the approach changes depending on the range of values assigned to the bit-sequences.